SISTEMA DE BANCA POR INTERNET BP

EJERCICIO PRACTICO

Autor: Paúl Aguilar

Rol: Arquitecto de Soluciones

Fecha: Marzo 2026

Versión: 1.0

Contenido

1. Introducción 2
2. Decisiones Arquitectónicas Clave y Justificaciones. 2
   2.1. Frontend Móvil: 2
   2.2. Frontend Web (SPA): 3
   2.3. Flujo de Autenticación OAuth 2.0 5
   2.4. Onboarding Biométrico 6
   2.5. Patrón para Clientes Frecuentes y Datos de Auditoría 7
   2.6. Patrón de Integración 8
   2.7. Estrategia de Resiliencia, Alta Disponibilidad y Monitoreo en Azure 9
   2.8. Comunicación Asíncrona mediante eventos 11
   2.9. Cache para Clientes: 12
3. Diagramas C4 de la Solución 13
   3.1. Diagrama de Contexto (Nivel 1) 13
   3.2. Diagrama de Contenedores (Nivel 2) 15
   3.3. Diagrama de Componentes (Nivel 3) 17
4. Arquitectura de Autenticación y Onboarding 19
5. Consideraciones Normativas para Entidades Financieras en Ecuador 20
6. Gestión de Costos en la Nube 20

1. Introducción

El presente documento detalla la arquitectura propuesta para el nuevo sistema de Banca por Internet de la entidad BP. El diseño se ha elaborado siguiendo el modelo C4 (Contexto, Contenedores, Componentes) y se enfoca en cumplir con los requerimientos funcionales (consultas, transferencias, pagos, onboarding biométrico) y no funcionales (alta disponibilidad, seguridad, escalabilidad, desacoplamiento) exigidos.

La solución se basa en un enfoque modular y escalable en la nube (Azure/AWS) con principios de alta disponibilidad, considerando las mejores prácticas en seguridad priorizando la experiencia del usuario multicanal (SPA y Móvil) y el cumplimiento normativo local ecuatoriano.

El sistema de banca por internet de BP permitirá a los clientes:

- Consultar histórico de movimientos.
- Realizar transferencias y pagos (propias e interbancarias).
- Recibir notificaciones obligatorias de cada transacción.
- Acceder desde una SPA web y una aplicación móvil multiplataforma.

2. Decisiones Arquitectónicas Clave y Justificaciones.

2.1. Frontend Móvil:

Opciones:

- **React Native:** Reutilización de lógica con la SPA (JavaScript/TypeScript) si la SPA está hecha en React; gran comunidad y librerías (especialmente para integración nativa biométrica).
- **Flutter:** Excelente rendimiento y UI consistente gracias a su motor de renderizado propio; creciente comunidad.

Estado: Aceptada

Contexto:

Se requiere desarrollar una aplicación móvil para banca por internet que funcione en iOS y Android. La empresa cuenta con presupuesto para desarrollo, pero busca optimizar recursos. La aplicación debe soportar funcionalidades nativas como cámara para reconocimiento facial, biometría (huella/rostro) y notificaciones push. El equipo de desarrollo tiene experiencia en JavaScript.

Decisión:

Se utilizará React Native como framework multiplataforma. Permite compartir lógica de negocio y validaciones con una posible SPA en React (ahorro de costes). Su madurez en integración con módulos nativos (cámara para reconocimiento facial, biometría del dispositivo) es superior y crítica para el onboarding.

Consecuencias:

- **Positivas:**
  - Reutilización de código entre iOS y Android (aproximadamente 90%).
  - Integración nativa fluida con módulos de cámara y biometría.
  - Amplia comunidad y librerías para banca/finanzas.
  - Hot reload que acelera el desarrollo.
  - Curva de aprendizaje corta para el equipo con experiencia en React.
- **Negativas:**
  - Performance ligeramente inferior a Flutter en animaciones complejas.
  - Dependencia de puentes JavaScript-Nativo para funcionalidades específicas.
  - Actualizaciones de iOS/Android pueden requerir actualizaciones del framework.
- **Riesgos:**
  - Cambios en políticas de App Store/Google Play que afecten el framework.
  - Necesidad de conocimiento nativo para problemas complejos.

2.2. Frontend Web (SPA):

Opciones:

- **React:** Librería flexible y eficiente para construir interfaces dinámicas; virtual DOM para buena experiencia de usuario.

- **Angular:** Framework completo con curva de aprendizaje más pronunciada, pero robusto para aplicaciones empresariales.

Estado: Aceptada.

Contexto:

Se necesita una aplicación web de página única (SPA) para banca por internet con alta interactividad, experiencia de usuario fluida y capacidad de integración con APIs REST. El mismo equipo desarrollará la web y la móvil, por lo que se busca alineación tecnológica.

Decisión:

Se utilizará **React** con TypeScript. Se alinea con la decisión del móvil (React Native), permitiendo compartir lógica de validación de formularios, tipos (TypeScript) y conocimientos del equipo, maximizando la cohesión y el desacoplamiento del negocio de la vista.

Consecuencias:

- **Positivas:**
    - Alineación con React Native (misma sintaxis, conceptos y estado).
    - Reutilización de lógica de negocio, validaciones y tipos (TypeScript) entre web y móvil.
    - Virtual DOM para alto rendimiento en actualizaciones de UI.
    - Ecosistema maduro de librerías para banca (formularios, gráficos, seguridad).
    - TypeScript añade tipado estático que reduce errores en producción.
- **Negativas:**
    - Angular ofrece más estructura "out-of-the-box" para equipos grandes.
    - Requiere configuración adicional para routing y estado global.
    - Curva de aprendizaje para desarrolladores sin experiencia en React.
- **Riesgos:**
    - Fragmentación del ecosistema React con nuevas versiones
    - Dependencia de librerías de terceros para funcionalidades core.

2.3. Flujo de Autenticación OAuth 2.0

Opciones:

- **Authorization Code + PKCE**
- Implicit

Estado: Aceptada

Contexto:

La empresa cuenta con un producto de identidad configurable (Azure AD B2C). Se requiere autenticar usuarios desde aplicaciones móviles y web SPA. El estándar OAuth 2.0 ofrece múltiples flujos, y se debe seleccionar el más seguro y apropiado para aplicaciones públicas (sin secreto de cliente confidencial).

Decisión:

Se implementará el flujo **Authorization Code con PKCE (Proof Key for Code Exchange)**.

Consecuencias:

- **Positivas:**
    - **Seguridad mejorada:** PKCE evita que un atacante intercepte el código de autorización y lo canjee, incluso sin secreto de cliente.
    - **Tokens de refresco:** Permite sesiones largas sin reautenticar, mejorando experiencia de usuario.
    - **Tokens no expuestos:** A diferencia del flujo Implicit, los tokens no viajan en la URL ni quedan en historial.
    - **Estándar IETF:** Recomendado oficialmente para aplicaciones nativas y SPA.
    - **Compatibilidad:** Azure AD B2C soporta PKCE nativamente.
- **Negativas:**
    - Mayor complejidad de implementación que el flujo Implicit.
    - Requiere gestión de code_verifier y code_challenge en el cliente.
    - Redirecciones adicionales pueden afectar percepción de velocidad.

- **Riesgos:**
    - Implementación incorrecta de PKCE puede generar vulnerabilidades.
    - Dependencia de la correcta configuración del proveedor de identidad.

2.4. Onboarding Biométrico

Opciones:

- **Servicio Especializado Externo (Ej. Onfido, Jumio):** Alta precisión, cumplen con normativas KYC/AML internacionales, reducen el tiempo de desarrollo y la responsabilidad sobre datos biométricos sensibles.
- **Desarrollo Propio:** Complejidad técnica y legal enorme, alto coste de mantenimiento.

Estado: Aceptada

Contexto:

El sistema requiere verificación de identidad para nuevos clientes mediante reconocimiento facial y validación de documentos de identidad. La normativa ecuatoriana (Ley de Datos Personales, Resoluciones de Junta Bancaria) exige altos estándares de seguridad y privacidad. Desarrollar una solución propia implicaría alto costo, complejidad y responsabilidad legal.

Decisión:

Se utilizará un servicio especializado de verificación de identidad: **Onfido**. Se integra vía API desde el backend. Este servicio se encarga de la verificación de identidad (documento vs selfie) y devuelve un resultado (aprobado/rechazado) + un token de referencia. Este token se usa para crear la cuenta del usuario, desacoplando la gestión de datos biométricos sensibles de nuestro sistema core.

Consecuencias:

- **Positivas:**
    - **Cumplimiento normativo:** Onfido cumple con KYC/AML internacional y facilita la auditoría
    - **Precisión:** Modelos de IA entrenados con millones de documentos, alta tasa de acierto
    - **Reducción de riesgo legal:** La empresa no almacena datos biométricos sensibles; Onfido retorna solo un token de verificación

    - **Tiempo de desarrollo:** Integración vía API, semanas vs meses de desarrollo propio
    - **Actualizaciones continuas:** El proveedor mejora constantemente los algoritmos contra fraudes
- **Negativas:**
    - **Costo por verificación:** Modelo de pago por uso que puede escalar con el crecimiento
    - **Dependencia externa:** Si el servicio falla, el onboarding se detiene
    - **Latencia de red:** La verificación depende de la conexión a internet del usuario y del servicio
    - **Privacidad de datos:** Transferencia de datos sensibles a un tercero (requiere consentimiento explícito)
- **Riesgos:**
    - Cambios en precios o términos del proveedor
    - Violación de seguridad en el proveedor que exponga datos de clientes
    - Necesidad de migrar si el proveedor cierra el servicio.

2.5. Patrón para Clientes Frecuentes y Datos de Auditoría

Opciones:

CQRS (Command Query Responsibility Segregation)

Estado: Aceptada

Contexto:

Se requiere: (1) una base de datos de auditoría que registre todas las acciones del cliente (requisito normativo), y (2) un mecanismo de persistencia de información para clientes frecuentes que permita consultas rápidas. El modelo tradicional CRUD en una sola base de datos podría generar contención y bajo rendimiento.

Decisión:

Se implementará el patrón CQRS (Command Query Responsibility Segregation).

Consecuencias:

- **Positivas:**

    - Separación de responsabilidades: Modelos de lectura (queries) y escritura (commands) independientes.
    - Optimización de consultas: El modelo de lectura para clientes frecuentes puede estar desnormalizado en Redis, con tiempos de respuesta < 10ms.
    - Auditoría inmutable: Cada comando genera un evento que se persiste en Cosmos DB, creando un registro trazable y a prueba de manipulaciones.
    - Escalabilidad independiente: El modelo de lectura puede escalar horizontalmente sin afectar la escritura.
    - Cumplimiento normativo: La auditoría basada en eventos satisface requisitos de la Junta Bancaria del Ecuador.
- **Negativas:**
    - Complejidad: Mayor que un CRUD tradicional; requiere sincronización entre modelos.
    - Consistencia eventual: El modelo de lectura puede tener un leve retraso respecto a la escritura.
    - Mantenimiento: Dos bases de datos (Redis y Cosmos DB) que requieren monitoreo.
- **Riesgos:**
    - Inconsistencias temporales entre el modelo de lectura y escritura.
    - Sobrecarga en la sincronización de eventos.

2.6. Patrón de Integración

Opciones:

API Gateway

Estado: Aceptada

Contexto:

La arquitectura contempla múltiples microservicios (Consultas, Transferencias, Onboarding, Notificaciones) que deben ser consumidos por aplicaciones web y móviles. Se requiere un punto único de entrada que maneje preocupaciones transversales como autenticación, rate limiting y monitoreo.

Decisión:
Se implementará un API Gateway utilizando Azure API Management.

Consecuencias:

- **Positivas:**
  - Punto único de entrada: Simplifica la comunicación para los clientes (una sola URL base).
  - Seguridad centralizada: Validación de tokens JWT en una sola capa, no en cada microservicio.
  - Cross-cutting concerns: Rate limiting, logging, caching y transformación de protocolos en un solo lugar.
  - Enrutamiento inteligente: Puede dirigir peticiones a diferentes versiones de microservicios.
  - Monitoreo unificado: Azure API Management proporciona analytics y alertas.
- **Negativas:**
  - Punto único de fallo: Si el gateway falla, todo el sistema se ve afectado (mitigado con HA).
  - Latencia adicional: Cada petición pasa por una capa extra.
  - Complejidad de configuración: Requiere definir políticas para cada ruta.
  - Costo: Azure API Management tiene costo adicional.
- **Riesgos:**
  - Configuración incorrecta que exponga microservicios internos.
  - Cuello de botella si no se escala adecuadamente.

2.7. Estrategia de Resiliencia, Alta Disponibilidad y Monitoreo en Azure

Estado: Aceptada

Contexto:

El sistema bancario debe garantizar disponibilidad 24/7 con mínima interrupción. La normativa ecuatoriana exige continuidad del servicio y protección de datos. Se cuenta con presupuesto para infraestructura en la nube (Azure).

Decisión:

Se implementará una arquitectura de Alta Disponibilidad (HA) multi-zona y Recuperación ante Desastres (DR) multi-región en Azure.

- **Alta Disponibilidad (HA) y Recuperación ante Desastres (DR):**
    - **Zonas de Disponibilidad:** Desplegar los microservicios, bases de datos (Azure SQL Managed Instance con ZRs) y Azure API Management en modo activo-activo a través de múltiples Zonas de Disponibilidad en una misma región (por ejemplo, US Este 2).
    - **DR:** Configurar una región secundaria (Failover Group para Azure SQL, Geo-Replication para Cosmos DB). En caso de desastre regional, se realiza un failover manual o automático. Azure Traffic Manager o Front Door se usarían para enrutar el tráfico a la región activa.
- **Tolerancia a Fallos y Self-Healing:**
    - **Azure Kubernetes Service (AKS):** Si se orquestan los microservicios con Kubernetes, esta plataforma se encarga de auto-reparar los pods fallidos, escalarlos y balancear la carga.
    - **Patrones de Resiliencia:** Implementar en los microservicios las librerías de Polly (Retry, Circuit Breaker, Timeout) para manejar fallos en las comunicaciones con el Core Bancario, que puede ser menos estable.
- **Monitoreo y Excelencia Operativa:**
    - **Azure Monitor & Application Insights:** Centralizar logs, métricas (rendimiento, latencia, errores) de toda la aplicación.
    - **Dashboards:** Crear paneles de control para el negocio (transacciones por minuto, usuarios activos) y para operaciones (estado de los servicios, uso de recursos).
    - **Alertas:** Configurar alertas proactivas (ej. latencia del Core > 2 seg, tasa de error > 1%, uso de CPU alto) que puedan disparar acciones de auto-escalado o notificar al equipo de operaciones.

Consecuencias

- **Positivas:**
    - Disponibilidad del 99.99%: Distribución en 3 zonas de disponibilidad dentro de la región primaria (East US 2).
    - Tolerancia a fallos: Si una zona falla, las otras continúan operando.

    - Recuperación ante desastres: Región secundaria (West US 2) con réplica activa-pasiva.
    - RTO < 15 minutos: Tiempo de recuperación objetivo.
    - RPO < 5 minutos: Pérdida máxima de datos en caso de desastre.
    - Azure Front Door: Enrutamiento inteligente y failover automático entre regiones.
- **Negativas:**
    - Costo: Aproximadamente 2.5x el costo de una arquitectura simple.
    - Complejidad: Configuración de replicación, failover y pruebas de recuperación.
    - Latencia entre regiones: La replicación de datos introduce latencia.
- **Riesgos:**
    - Failover automático puede activarse por falsas alarmas.
    - Pruebas de DR deben realizarse periódicamente sin afectar producción.

2.8. Comunicación Asíncrona mediante eventos

Estado: Aceptada

Contexto:

Se requiere notificar a clientes sobre movimientos (por normativa) utilizando al menos dos canales. Además, se necesita auditoría de todas las transacciones. La comunicación síncrona entre microservicios crearía acoplamiento y afectaría la disponibilidad.

Decisión:

Se implementará un **bus de eventos** utilizando **Azure Event Grid** para comunicación asíncrona.

Consecuencias:

- **Positivas:**
    - **Desacoplamiento:** El microservicio de Transferencias publica eventos sin conocer a los consumidores.

    - **Escalabilidad:** Múltiples suscriptores pueden escalar independientemente.
    - **Resiliencia:** Si un suscriptor falla, los eventos no se pierden (dead-lettering).
    - **Múltiples canales:** Notificaciones SMS, Email y Push pueden reaccionar al mismo evento.
    - **Auditoría automática:** Un suscriptor de auditoría persiste todos los eventos en Cosmos DB.

- **Negativas:**
    - **Complejidad:** Mayor que llamadas síncronas; requiere manejo de idempotencia.
    - **Consistencia eventual:** Los suscriptores pueden procesar eventos con retraso.
    - **Depuración:** Trazar flujos asíncronos es más complejo.
    - **Costo por operación:** Event Grid tiene costo por millón de operaciones.
- **Riesgos:**
    - Duplicación de eventos si no se implementa idempotencia.
    - Orden de eventos no garantizado (requiere lógica adicional).

2.9. Cache para Clientes:

Estado: Aceptada

Contexto:

Los clientes frecuentes consultan su saldo y últimos movimientos repetidamente. Consultar directamente al Core Bancario cada vez genera alto costo y latencia. Se necesita una solución de baja latencia (< 50ms) para mejorar experiencia de usuario.

Decisión:

Se implementará **Azure Redis Cache** para almacenar datos desnormalizados de clientes frecuentes.

Consecuencias:

- **Positivas:**
  - **Latencia ultra baja:** Respuestas < 5ms para datos en cache
  - **Reducción de carga:** Disminuye consultas al Core Bancario en ~80% para clientes frecuentes
  - **Escalabilidad:** Redis cluster puede manejar millones de operaciones por segundo
  - **Datos desnormalizados:** Almacena último saldo, últimos 10 movimientos, contactos frecuentes
  - **Actualización por eventos:** Los cambios se reflejan en cache mediante suscripción a Event Grid
- **Negativas:**
  - **Consistencia eventual:** Puede haber un pequeño retraso entre la transacción real y la cache
  - **Costo adicional:** Redis Cache tiene costo por capacidad y replicación
  - **Complejidad:** Estrategia de invalidación de cache (TTL, eventos)
  - **Memoria volátil:** Los datos se pierden si no hay persistencia (mitigado con replicación)
- **Riesgos:**
  - Datos obsoletos si la sincronización falla
  - Evicción de datos si la cache se llena

3. Diagramas C4 de la Solución

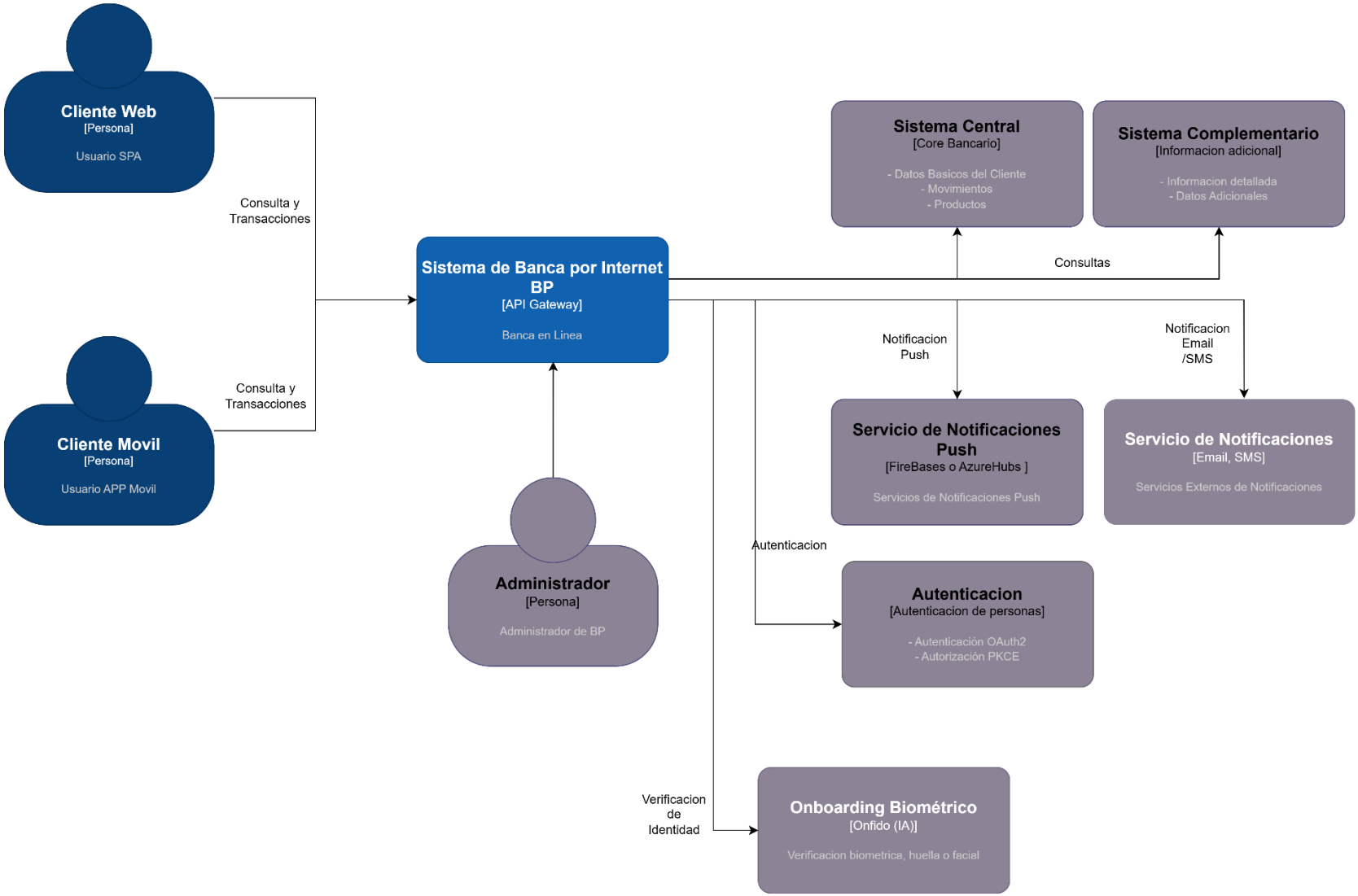
3.1. Diagrama de Contexto (Nivel 1)

Audiencia: Usuarios no técnicos (Dirección, Producto).

Descripción: Muestra el sistema de Banca por Internet (BP) como una caja negra y cómo interactúa con los actores externos y sistemas.

Diagrama de Contexto - Sistema de Banca por Internet BP

C4 Nivel 1



- **Actor Principal: Cliente** (Persona que usa la SPA o App Móvil para consultar y operar).
- **Actor Secundario: Administrador** (Gestiona monitoreo y configuración).
- **Sistemas Externos:**
    - **Core Bancario:** Sistema de registro (Fuente de verdad para datos de cliente, saldos y movimientos históricos).
    - **Sistema de Información Complementaria:** Provee datos detallados del cliente (ej. información de contacto detallada, productos no financieros).
    - **Sistema de Notificaciones (Interno/Externo):** Envía emails y SMS.
    - **Proveedor de Notificaciones Push:** (Ej. Azure Notification Hubs, Firebase Cloud Messaging) para notificaciones en tiempo real a la App Móvil.
    - **Servicio de Onboarding Biométrico (Onfido):** Verifica la identidad del cliente mediante IA.
    - **Autoridad Certificadora (Azure AD B2C):** Gestiona el registro, inicio de sesión y emisión de tokens OAuth2.

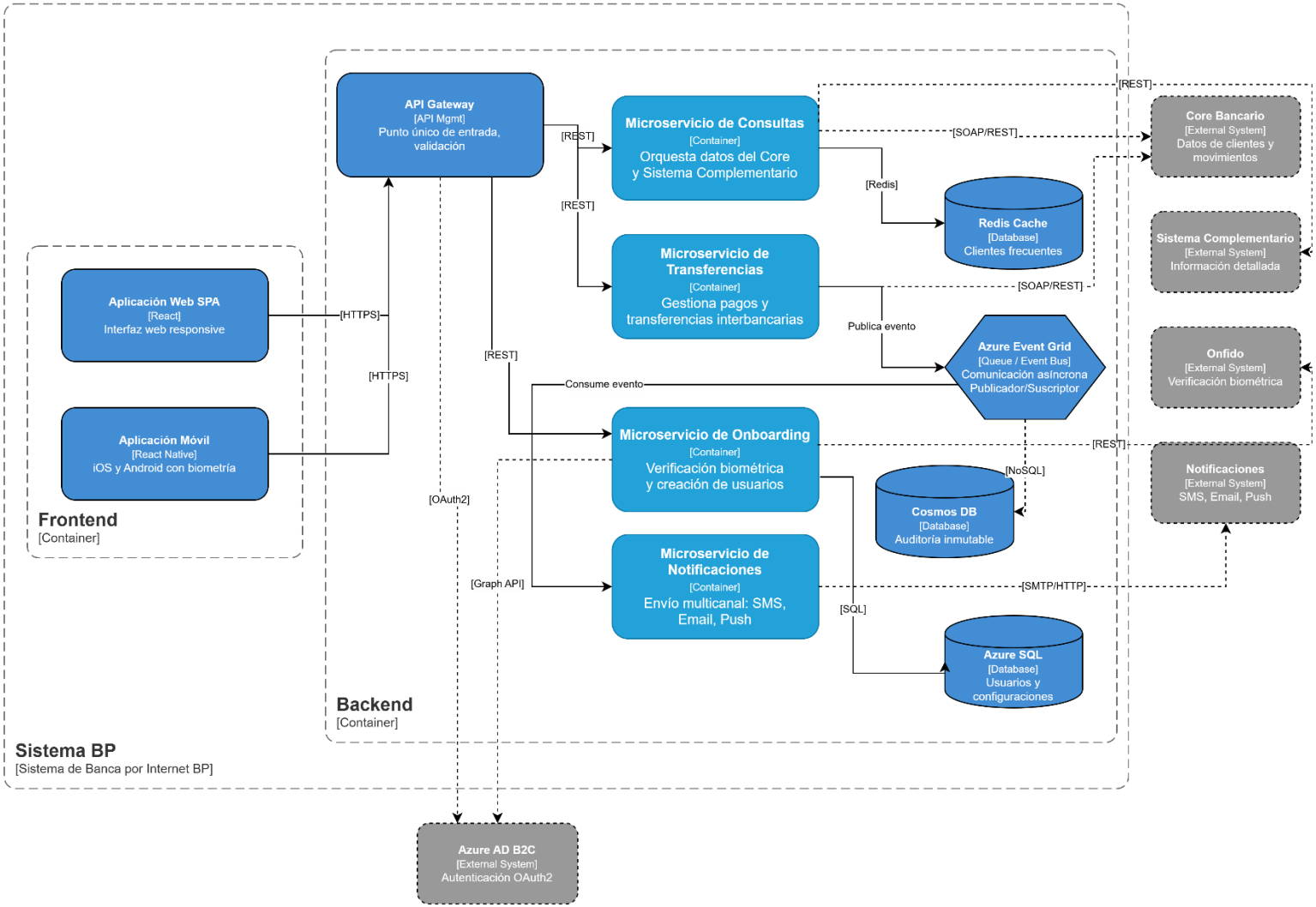
3.2. Diagrama de Contenedores (Nivel 2)

Audiencia: Técnicos (Desarrolladores, DevOps).

Descripción: Muestra las aplicaciones y almacenes de datos de alto nivel que componen el sistema BP.

Diagrama de Contenedores- Sistema de Banca por Internet BP

C4 Nivel 2



Leyenda:

Línea sólida: Comunicación síncrona (REST/HTTPS)

Línea punteada: Comunicación asíncrona / Sistemas externos

- **Contenedores de Frontend:**
    - **Aplicación Web SPA (React):** Alojada en Azure Static Web Apps o CDN. Interfaz web dinámica.
    - **Aplicación Móvil (React Native):** Distribuida vía App Store/Google Play.
- **Contenedores Backend:**
    - **API Gateway (Azure API Management):** Punto de entrada único. Gestiona autenticación, y enrutamiento.
    - **Servicio de Autorización (Azure AD B2C):** Producto externo configurado para OAuth2. Gestiona el ciclo de vida de usuarios y tokens.
    - **Microservicio de Consultas (ASP.NET Core / Spring Boot):** Orquesta la obtención de datos desde el Core y el Sistema Complementario. Alimenta la cache de lecturas.
    - **Microservicio de Transferencias:** Gestiona la lógica de negocio para pagos y transferencias (propias e interbancarias). Escribe en el Core y publica eventos.
    - **Microservicio de Onboarding:** Orquesta el flujo de registro, integración con Onfido y creación de usuario en Azure AD B2C.
    - **Microservicio de Notificaciones:** Consume eventos del bus y envía notificaciones por los diferentes canales (SMS, Email, Push).
- **Contenedores de Datos:**
    - **Base de Datos de Auditoria (Azure Cosmos DB):** Almacena eventos inmutables de todas las transacciones.
    - **Cache de Lectura (Azure Redis Cache):** Almacena datos desnormalizados de clientes frecuentes para consultas de bajo coste y alta velocidad.
    - **Base de Datos Operacional (Azure SQL / PostgreSQL):** Persistencia propia de cada microservicio (si requiere estado).

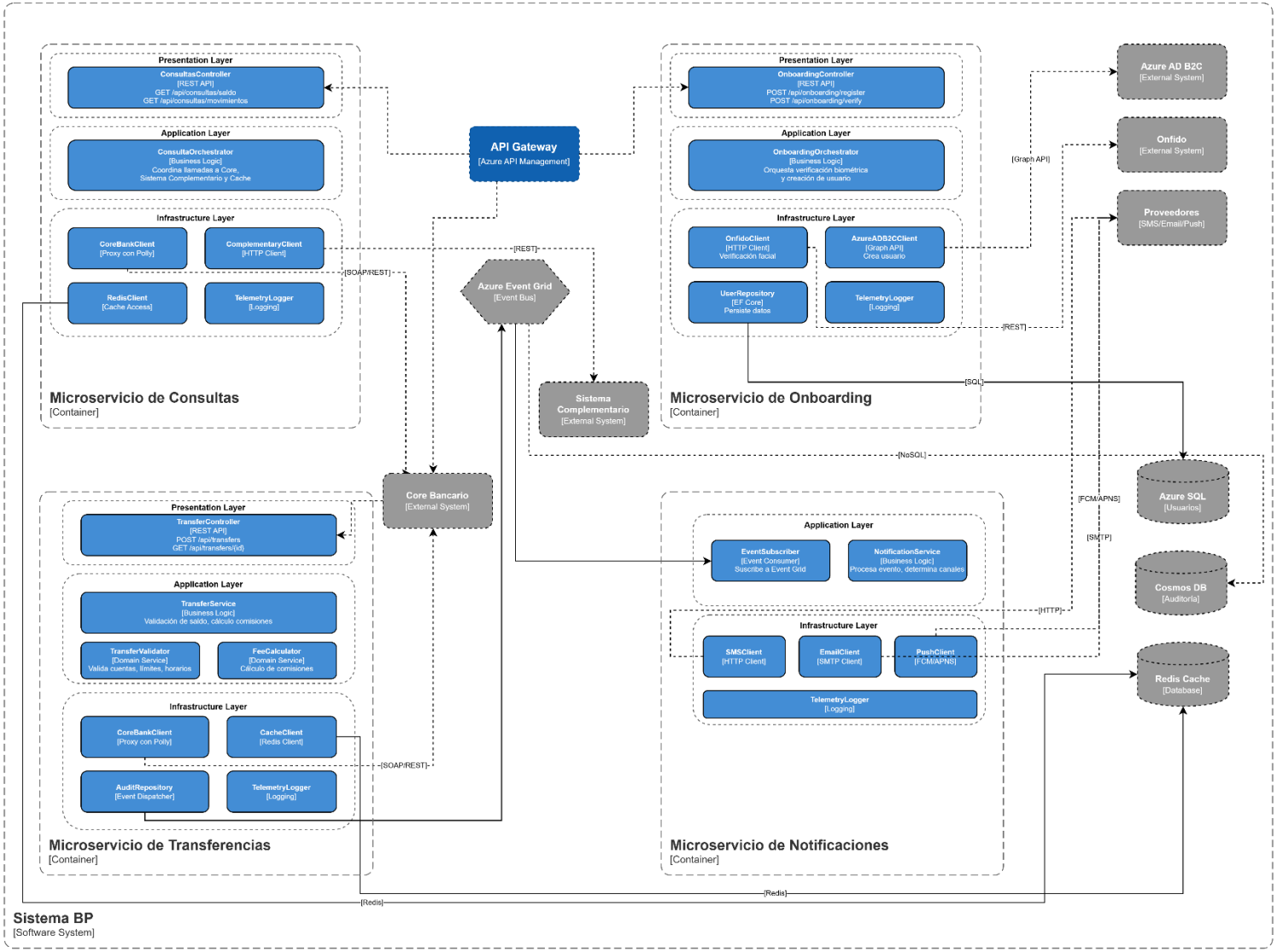
3.3. Diagrama de Componentes (Nivel 3)

Audiencia: Arquitectos y Desarrolladores Senior.

Descripción: Profundiza en el interior de un contenedor clave, por ejemplo, el **Microservicio de Transferencias**, para mostrar cómo está estructurado y cómo se comunica.

Diagrama de Componentes - Sistema de Banca por Internet BP

C4 Nivel 3



Leyenda:

Línea sólida: Comunicación síncrona (REST/HTTPS)

Línea punteada: Comunicación asíncrona / Sistemas externos

- **Componentes Internos:**
  - **API Controller (REST):** Expone endpoints /api/transfers. Valida el token JWT (ya validado por el Gateway).
  - **Transfer Service (Lógica de Negocio):** Contiene las reglas para validar saldo (consultando al Core), aplicar comisiones, etc.
  - **Core Bank Client (Proxy):** Componente que encapsula la comunicación (vía SOAP o REST) con el sistema Core Bancario. Implementa patrones de resiliencia (Retry, Circuit Breaker con Polly).
  - **Audit Repository (Event Dispatcher):** En lugar de escribir directamente en la BD de auditoría, este componente publica un evento TransferCompleted en **Azure Event Grid**.
- **Comunicación:**
  - **Síncrona (REST/HTTPS):** Para la comunicación entre el microservicio y el Core (alto coste, requiere respuesta inmediata).
  - **Asíncrona (Eventos):** Para la notificación al cliente y el registro de auditoría. El microservicio publica el evento y se desentiende. El **Microservicio de Notificaciones** y un **Suscriptor de Auditoría** (otro componente) consumen este evento de forma independiente, garantizando el desacoplamiento.

4. Arquitectura de Autenticación y Onboarding

1. **Registro (Onboarding):**
   - El usuario inicia el proceso en la App Móvil.
   - La App se comunica con el **Microservicio de Onboarding**.
   - Este servicio invoca al **Proveedor Biométrico (Onfido)** y recibe un applicant_id y un resultado.
   - Si es exitoso, el **Microservicio de Onboarding** utiliza las APIs de **Azure AD B2C** para crear un nuevo usuario.
   - Se registra el evento en la base de datos de auditoría.
2. **Inicio de Sesión y Autenticación:**
   - El usuario (en SPA o Móvil) inicia el flujo **OAuth 2.0 Authorization Code + PKCE**.
   - Es redirigido a los endpoints de **Azure AD B2C**.
   - AD B2C presenta la interfaz de login (que puede ser personalizada con la marca de BP).

- Tras autenticar (usuario/contraseña, y opcionalmente un segundo factor), AD B2C genera un **Código de Autorización**.
- La aplicación canjea este código (junto con el PKCE) por un **Access Token (JWT)** y un **Refresh Token**.
- Para inicios posteriores, la App Móvil puede usar el SDK de AD B2C para lanzar la autenticación biométrica del dispositivo (huella/rostro), que canjeará silenciosamente por un nuevo token si es válido.

5. Consideraciones Normativas para Entidades Financieras en Ecuador

- **Ley de Protección de Datos Personales (Ecuador):** La arquitectura debe garantizar la confidencialidad, integridad y disponibilidad de los datos personales. El uso de un servicio externo para biometría (Onfido) ayuda a transferir la responsabilidad del almacenamiento de datos sensibles. El cifrado en reposo (Azure Storage Service Encryption) y en tránsito (TLS 1.2+) es obligatorio. Se debe implementar un consentimiento explícito para el tratamiento de datos biométricos.
- **Resolución JB-2012-2148 (Junta Bancaria):** Normativa de Gestión de Seguridad de la Información. Exige la implementación de controles de acceso, autenticación reforzada (2FA) para transacciones sensibles, y auditoría de eventos. Nuestra arquitectura cumple con OAuth2, MFA desde AD B2C, y la base de datos de auditoría inmutable.
- **Circular SB-2011-659 (Seguridad de Transacciones):** Requiere la notificación al cliente de cada movimiento. El **Microservicio de Notificaciones** y el bus de eventos garantizan este envío por al menos dos canales (Push y SMS, por ejemplo).
- **Resolución sobre KYC (Conozca a su Cliente):** El proceso de onboarding con verificación biométrica y de documento de identidad satisface los requisitos de identificación positiva del cliente.

6. Gestión de Costos en la Nube

- **Optimización:**
  - Usar **Azure Reservations** para instancias de VM o AKS con carga base predecible.
  - Implementar **auto-escalado** en los microservicios para que los recursos consumidos se ajusten a la demanda (escala a cero fuera de horas pico si es posible).

- Elegir el nivel adecuado de servicio: Por ejemplo, usar **Cosmos DB (no relacional) Serverless** para la base de datos de auditoría si la carga es esporádica.
- Utilizar el **Azure Pricing Calculator** para modelar el coste antes del despliegue y etiquetar recursos para un chargeback/showback preciso a las unidades de negocio.

- **Modelo de Costos por Entorno**

Entorno	Configuración	Estrategia
Desarrollo	1 nodo B2s, SQL Serverless, sin HA	Spot instances, auto-pausa nocturna
QA	2 nodos D2s v3, HA básica	Reserved 1 año
Producción	3+ nodos D8s v3 multi-zona, HA completa	Reserved 3 años + optimización
DR	Región secundaria pasiva	Solo datos críticos replicados

- **Conclusión**

  La arquitectura propuesta en Azure permite una gestión granular de costos mediante:

  - Modelo de pago por uso con auto-escalado para adaptarse a la demanda.
  - Reserved Instances para cargas de trabajo predecibles (producción).
  - Serverless para componentes con demanda variable (auditoría, desarrollo).
  - Lifecycle policies para optimizar almacenamiento de datos históricos.
  - Tagging para visibilidad y chargeback por unidad de negocio.